

1. SELECT y Fundamentos Básicos

La sentencia **SELECT** es el pilar de SQL. Se utiliza para extraer datos de una tabla. Puedes especificar columnas concretas separadas por comas o usar un asterisco (*) para traer todas las columnas.

Para evitar duplicados y ver solo valores únicos, se antepone **DISTINCT**. La cláusula **LIMIT** restringe el número máximo de filas devueltas.

```
SELECT DISTINCT country
FROM customers
LIMIT 10;
```

2. Paginación (LIMIT, OFFSET y TOP)

Para fragmentar listados gigantescos, en PostgreSQL/MySQL se usa **LIMIT** (cuántos registros) junto a **OFFSET** (cuántos te saltas desde el principio).

Para simular la página 3 mostrando 10 elementos, te saltas 20 registros y pides los 10 siguientes. En SQL Server se usa **TOP N**.

```
-- Página 3 (registros del 21 al 30)
SELECT * FROM products
ORDER BY id
LIMIT 10 OFFSET 20;

-- En SQL Server
SELECT TOP 10 * FROM products;
```

3. Filtrado Avanzado (WHERE y LIKE)

La cláusula **WHERE** filtra las filas antes de cualquier cálculo. Se combina con operadores lógicos (**AND**, **OR**), rangos (**BETWEEN**) o listas (**IN**).

Para textos parciales se usa **LIKE**: % sustituye a cualquier secuencia de caracteres y _ a un único carácter.

```
SELECT * FROM users
WHERE email LIKE '%@gmail.com'
AND code LIKE 'A_12%';
```

4. Manipulación de Textos

Funciones clave para transformar cadenas:

- **LENGTH(str)**: Cuenta caracteres totales.
- **UPPER / LOWER**: Cambian mayúsculas/minúsculas.
- **POSITION(sub IN str)**: Devuelve la posición numérica exacta de un carácter (ej. la arroba @).
- **SUBSTRING(str FROM start FOR len)**: Corta un trozo de texto específico.

```
SELECT email,
SUBSTRING(email FROM 1 FOR POSITION('@' IN
email) - 1) AS username
FROM users;
```

5. Agregación y Orden de Ejecución

Las funciones (**SUM**, **AVG**, **COUNT**) resumen filas. Al agrupar con **GROUP BY**, los filtros sobre grupos van en **HAVING**, no en el **WHERE**.

Orden de ejecución:

1. **FROM / JOIN**
2. **WHERE**
3. **GROUP BY**
4. **HAVING**
5. **SELECT**
6. **ORDER BY / LIMIT**.

6. Funciones de Ventana (Window Functions) 🌞

Calculan métricas analíticas **manteniendo intactas todas las filas originales** de la tabla (a diferencia del **GROUP BY**). Se estructuran con la cláusula **OVER()**:

- **PARTITION BY**: Divide la tabla en compartimentos independientes.
- **ORDER BY**: Ordena internamente las filas dentro de cada partición.

```
SELECT name, department, salary,
AVG(salary) OVER(PARTITION BY department) AS
avg_dept_sal
FROM employees;
```

7. Rankings y Puestos

Diferencias clave entre funciones de ranking:

- **ROW_NUMBER()**: Numera de forma estrictamente secuencial sin huecos (1, 2, 3...).
- **RANK()**: Asigna el mismo número en empates pero **deja huecos** en el siguiente (1, 2, 2, 4).
- **DENSE_RANK()**: Similar a RANK pero **sin dejar huecos** (1, 2, 2, 3).

```
SELECT name, salary,
ROW_NUMBER() OVER(ORDER BY salary DESC) AS r_num,
RANK() OVER(ORDER BY salary DESC) AS rnk,
DENSE_RANK() OVER(ORDER BY salary DESC) AS
dense_rnk
FROM employees;
```

8. Subconsultas y CTEs

Una **CTE** (definida con **WITH**) actúa como una vista virtual temporal durante la ejecución, permitiendo descomponer consultas complejas en bloques legibles.

```
WITH Dept_Summary AS (
SELECT department, AVG(salary) AS avg_sal
FROM employees GROUP BY department
)
SELECT e.name, e.salary, d.avg_sal
FROM employees e
JOIN Dept_Summary d ON e.department = d.department;
```